



**UNIVERSIDADE FEDERAL DO RURAL DO SEMI-ÁRIDO
CENTRO MULTIDISCIPLINAR DE PAU DOS FERROS
DEPARTAMENTO DE ENGENHARIAS E TECNOLOGIA
PROGRAMAÇÃO CONCORRENTE E DISTRIBUÍDA
DOCENTE: ÍTALO AUGUSTO SOUZA DE ASSIS**

Allyson Bruno de Freitas Fernandes

PROGRAMAÇÃO CONCORRENTE E DISTRIBUÍDA

PAU DOS FERROS –
RN 2026

QUESTÕES RESPONDIDAS

QUESTÃO 01 - RESPONDIDO

QUESTÃO 02 - RESPONDIDO

QUESTÃO 03 - RESPONDIDO

QUESTÃO 04 - RESPONDIDO

QUESTÃO 05 - RESPONDIDO

QUESTÃO 06 - RESPONDIDO

QUESTÃO 07 - RESPONDIDO

QUESTÃO 08 - RESPONDIDO

QUESTÃO 09 - RESPONDIDO

QUESTÃO 10 - RESPONDIDO

QUESTÃO 11 - RESPONDIDO

QUESTÃO 12 - NÃO RESPONDIDO

QUESTÃO 13 - NÃO RESPONDIDO

QUESTÃO 14 - RESPONDIDO

Programação de memória distribuída com MPI

1. **Modifique a regra trapezoidal para que ela estime corretamente a integral mesmo que `comm_sz` não divida `n` uniformemente. Você ainda pode assumir que $n \geq \text{comm_sz}$.**

O problema no cálculo da integral quando `comm_sz` não é divisível por “`n`” acontece, pois há trapézios que não são considerados no cálculo, para resolver isso, são distribuídas fatias a mais fatias essas que não seriam incluídas no cálculo para os processos iniciais de maneira uniforme.

Para isso é feita uma diferença no cálculo do `local_n`, `local_a` e consequentemente no `local_b` quando o `comm_sz` não é divisível pelo número de trapézios.

```
C/C++
h = (b - a) / n;

int base_n    = n / comm_sz;
int remainder = n % comm_sz;

if (my_rank < remainder) {
    local_n = base_n + 1;
    local_a = a + my_rank * local_n * h;
} else {
    local_n = base_n;
    local_a = a + (my_rank * base_n + remainder) * h;
}
local_b = local_a + local_n * h;

local_int = Trap(local_a, local_b, local_n, h);
```

2. **Modifique o programa que apenas imprime uma linha de saída de cada processo (`mpi_output.c`) para que a saída seja impressa na ordem de classificação do processo: processe a saída de 0 primeiro, depois processe 1 e assim por diante.**

Uma solução é usar um "token" passado sequencialmente de processo em processo.

- Processo 0 imprime e envia o token para o processo 1.
- Cada processo `q` ($q > 0$) espera receber o token do processo `q-1`, imprime sua mensagem e repassa o token para `q+1`.

```
C/C++
if (my_rank == 0) {
    printf("Proc %d > ...\n", my_rank);
    fflush(stdout);
    MPI_Send(&token, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
} else {
```

```

    MPI_Recv(&token, 1, MPI_INT, my_rank - 1, 0, MPI_COMM_WORLD,
&status);
    printf("Proc %d > ...\n", my_rank);
    fflush(stdout);
    if (my_rank < comm_sz - 1)
        MPI_Send(&token, 1, MPI_INT, my_rank + 1, 0,
MPI_COMM_WORLD);
}

```

3. Suponha que um programa seja executado com $comm_sz$ processos e que x seja um vetor com n componentes. Como os componentes de x seriam distribuídos entre os processos em um programa que usasse uma distribuição:

(a) em bloco?

Cada processo recebe um bloco contíguo de componentes.
Seja $base = n / p$ e $rem = n \% p$

Processo q recebe:

- $local_n = base + 1$, se $q < rem$
- $local_n = base$, se $q \geq rem$

Índice inicial do processo q :

- $first = q * (base + 1)$, se $q < rem$
- $first = q * base + rem$, se $q \geq rem$

Um exemplo seria $n=10$ e $p=3$

Processo	Componente
0	0,1,2,3
1	4,5,6,7
2	8,9

(b) cíclica?

Os componentes são distribuídos em round-robin: componente i vai para o processo $i \% p$.

Processo q recebe os índices: $q, q+p, q+2p, q+3p, \dots$

$local_n$ do processo q :

- $local_n = n/p + 1$, se $q < n\%p$
- $local_n = n/p$, se $q \geq n\%p$

Exemplo $n=10$ e $p=3$

Processo	Componente
0	x[0],x[3],x[6],x[9]
1	x[1],x[4],x[7]
2	x[2],x[5],x[8]

(c) bloco-cíclica com tamanho de bloco b?

Os componentes são agrupados em blocos de tamanho b e distribuídos ciclicamente.

Bloco j (0-indexed) corresponde a índices globais $[j*b, j*b+1, \dots, j*b+b-1]$.

O bloco j vai para o processo $j \% p$.

Total de blocos completos: $\text{num_blocks} = n / b$

Blocos sobrando que não completam b: pode haver um bloco parcial de tamanho $n \% b$.

Processo q recebe os blocos: q, q+p, q+2p, ...

Para garantir distribuição justa (diferença máxima de 1 entre processos):

$\text{total_blocks} = \text{ceil}(n / b)$

$\text{base_blocks} = \text{total_blocks} / p$

$\text{rem_blocks} = \text{total_blocks} \% p$

Processo q recebe:

- $\text{base_blocks} + 1$ blocos, se $q < \text{rem_blocks}$
- base_blocks blocos, se $q \geq \text{rem_blocks}$

O último bloco pode ser parcial (tamanho $n \% b$, se $!= 0$).

Exemplo $n=16$, $p=3$ e $b=2$

Processo	Componente
0	0,1,6,7,12,13
1	2,3,8,9,14,15
2	4,5,10,11

Suas respostas devem ser genéricas para que possam ser usadas independentemente dos valores de `comm_sz`, `n` e `b`. Ao mesmo tempo, as distribuições apresentadas nas respostas devem ser "justas", de modo que, se q e r forem dois processos quaisquer, a diferença entre o número de componentes atribuídos a q e a r seja a menor possível.

4. Escreva um programa MPI que receba do usuário dois vetores e um escalar, todos lidos pelo processo 0 e distribuídos entre os processos. O primeiro vetor deve ser multiplicado pelo escalar. Para o segundo vetor, deve-se calcular a sua norma. Os resultados calculados devem ser coletados no processo 0, que os imprime. Você pode assumir que `n`, a ordem dos vetores, é divisível por `comm_sz`.

C/C++

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <mpi.h>

int main(void) {
    int my_rank, comm_sz, n, local_n;
    double scalar;
    double *x = NULL, *y = NULL;           /* vetores globais
(apenas proc 0) */
    double *local_x, *local_y;             /* vetores locais */
    double *result_x = NULL;               /* resultado global de
x*scalar */
    double local_norm_sq, global_norm_sq; /* para norma de y */

    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

    /* Processo 0 lê os dados */
    if (my_rank == 0) {
        printf("Digite n (ordem dos vetores, divisível por %d): ",
comm_sz);
        scanf("%d", &n);
        printf("Digite o escalar: ");
        scanf("%lf", &scalar);

        x = malloc(n * sizeof(double));
        y = malloc(n * sizeof(double));

        printf("Digite os %d elementos do vetor x:\n", n);
        for (int i = 0; i < n; i++) scanf("%lf", &x[i]);

        printf("Digite os %d elementos do vetor y:\n", n);
        for (int i = 0; i < n; i++) scanf("%lf", &y[i]);
    }

    /* Broadcast de n e scalar */
    MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(&scalar, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    local_n = n / comm_sz;
    local_x = malloc(local_n * sizeof(double));
    local_y = malloc(local_n * sizeof(double));

    /* Distribui os vetores */
    MPI_Scatter(x, local_n, MPI_DOUBLE, local_x, local_n,
MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Scatter(y, local_n, MPI_DOUBLE, local_y, local_n,
```

```

MPI_DOUBLE, 0, MPI_COMM_WORLD);

/* Operação 1: multiplica local_x pelo escalar */
for (int i = 0; i < local_n; i++)
    local_x[i] *= scalar;

/* Operação 2: soma parcial dos quadrados para a norma de y */
local_norm_sq = 0.0;
for (int i = 0; i < local_n; i++)
    local_norm_sq += local_y[i] * local_y[i];

/* Coleta vetor x*scalar no processo 0 */
if (my_rank == 0) result_x = malloc(n * sizeof(double));
MPI_Gather(local_x, local_n, MPI_DOUBLE, result_x, local_n,
MPI_DOUBLE, 0, MPI_COMM_WORLD);

/* Reduz a soma dos quadrados para a norma */
MPI_Reduce(&local_norm_sq, &global_norm_sq, 1, MPI_DOUBLE,
MPI_SUM, 0, MPI_COMM_WORLD);

/* Processo 0 imprime resultados */
if (my_rank == 0) {
    printf("\nResultado de x * %.2f:\n", scalar);
    for (int i = 0; i < n; i++) printf("%.4f ", result_x[i]);
    printf("\n");
    printf("Norma de y: %.6f\n", sqrt(global_norm_sq));

    free(x); free(y); free(result_x);
}

free(local_x);
free(local_y);
MPI_Finalize();
return 0;
}

```

5. Encontrar somas de prefixos é uma generalização da soma global. Em vez de simplesmente encontrar a soma de n valores,

$$x_0 + x_1 + \dots + x_{n-1}, (1)$$

as somas dos prefixos são as n somas parciais

$$x_0, x_0 + x_1, x_0 + x_1 + x_2, \dots, x_0 + x_1 + \dots + x_{n-1}. (2)$$

(a) Elabore um algoritmo serial para calcular as n somas de prefixos de um vetor com n elementos.

C/C++

```
int vector[8] = {5, 1, 8, 4, 3, 6, 4, 3};
int sum_prefixs[8];
for (int i = 0; i < 8; i++){
    sum_prefixs[i] = vector[i];
    if (i > 0)
        sum_prefixs[i] += sum_prefixs[i-1];
}
```

(b) Suponha $n = 2k$ para algum inteiro positivo k . Crie um algoritmo paralelo para um sistema com n processos, cada um armazenando um dos elementos de x , que exija apenas k fases de comunicação.

- Sem utilizar MPI_Scan

C/C++

```
/*
 * Compile: mpicc -g -Wall -o mpi_prefix_b mpi_prefix_b.c
 * Run:      mpiexec -n <p> ./mpi_prefix_b    (p deve ser potência
de 2)
 */
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char* argv[]) {
    int my_rank, comm_sz;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

    /* Cada processo gera um valor (semente diferente por rank) */
    srand(my_rank + 42);
    double my_val = (double)(rand() % 100);

    double cur = my_val;
    for (int half = 1; half < comm_sz; half <= 1) {
        double received = 0.0;
        if (my_rank + half < comm_sz)
            MPI_Send(&cur, 1, MPI_DOUBLE, my_rank + half, 0,
MPI_COMM_WORLD);
        if (my_rank - half >= 0)
            MPI_Recv(&received, 1, MPI_DOUBLE, my_rank - half, 0,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        cur += received;
        MPI_Barrier(MPI_COMM_WORLD);
    }

    if (my_rank == 0) {
```



```

        printf("Proc %d: valor=%.0f prefix_sum=%.0f\n", my_rank,
my_val, cur);
        fflush(stdout);
        if (comm_sz > 1) {
            int token = 1;
            MPI_Send(&token, 1, MPI_INT, 1, 99, MPI_COMM_WORLD);
        }
    } else {
        int token;
        MPI_Recv(&token, 1, MPI_INT, my_rank - 1, 99,
            MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Proc %d: valor=%.0f prefix_sum=%.0f\n", my_rank,
my_val, cur);
        fflush(stdout);
        if (my_rank < comm_sz - 1)
            MPI_Send(&token, 1, MPI_INT, my_rank + 1, 99,
MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}

```

(c) O MPI fornece uma função de comunicação coletiva, MPI_Scan, que pode ser usada para calcular somas de prefixos:

```

int MPI_Scan(
void* sendbuf_p /* in */,
void* recvbuf_p /* out */,
int count /* in */,
MPI_Datatype datatype /* in */,
MPI_Op op /* in */,
MPI_Comm comm /* in */);

```

Ela opera em arrays com count elementos; sendbuf_p e recvbuf_p devem se referir a blocos de count elementos do tipo datatype. O argumento op é igual ao op para o MPI_Reduce. Escreva um programa MPI que gere um vetor aleatório de count elementos em cada processo MPI, encontre as somas dos prefixos e imprima os resultados.

C/C++

```

/*
 * Compile: mpicc -g -Wall -o mpi_prefix_c mpi_prefix_c.c
 * Run:      mpiexec -n <p> ./mpi_prefix_c
 */
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

```

```

int main(int argc, char* argv[]) {
    int my_rank, comm_sz;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

    srand(my_rank + 42);
    double my_val = (double)(rand() % 100);

    double scan_result;
    MPI_Scan(&my_val, &scan_result, 1, MPI_DOUBLE, MPI_SUM,
MPI_COMM_WORLD);

    if (my_rank == 0) {
        printf("Proc %d: valor=%.0f prefix_sum=%.0f\n", my_rank,
my_val, scan_result);
        fflush(stdout);
        if (comm_sz > 1) {
            int token = 1;
            MPI_Send(&token, 1, MPI_INT, 1, 99, MPI_COMM_WORLD);
        }
    } else {
        int token;
        MPI_Recv(&token, 1, MPI_INT, my_rank - 1, 99,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Proc %d: valor=%.0f prefix_sum=%.0f\n", my_rank,
my_val, scan_result);
        fflush(stdout);
        if (my_rank < comm_sz - 1)
            MPI_Send(&token, 1, MPI_INT, my_rank + 1, 99,
MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}

```

6. Uma alternativa para um allreduce estruturado em borboleta é uma estrutura de passagem em anel. Em uma passagem de anel, se houver p processos, cada processo q envia dados para o processo $q + 1$, exceto que o processo $p - 1$ envia dados para o processo 0. Isso é repetido até que cada processo tenha o resultado desejado. Assim, podemos implementar allreduce com o seguinte código:

```

sum = temp_val = my_val;
for (i = 1; i < p; i++) {
    MPI_Sendrecv_replace(&temp_val, 1, MPI_INT, dest,

```

```

        sendtag, source, recvtag, comm, &status);
        sum += temp_val;
    }
}

```

(a) Escreva um programa MPI que implemente esse algoritmo para o allreduce. Como seu desempenho se compara ao allreduce estruturado em borboleta?

C/C++

```

/*
 * Compile: mpicc -g -Wall -o mpi_ring_a mpi_ring_a.c
 * Run:     mpiexec -n <p> ./mpi_ring_a
 */
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char* argv[]) {
    int my_rank, comm_sz;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

    srand(my_rank + 10);
    int my_val = rand() % 20;

    int dest    = (my_rank + 1) % comm_sz;
    int source  = (my_rank - 1 + comm_sz) % comm_sz;
    int sendtag = 0, recvtag = 0;

    int sum = my_val;
    int temp_val = my_val;

    for (int i = 1; i < comm_sz; i++) {
        MPI_Sendrecv_replace(&temp_val, 1, MPI_INT,
                             dest, sendtag,
                             source, recvtag,
                             MPI_COMM_WORLD, &status);

        sum += temp_val;
    }

    printf("Proc %d: my_val=%d sum=%d\n", my_rank, my_val, sum);
    fflush(stdout);

    MPI_Finalize();
    return 0;
}

```

(b) Modifique o programa MPI que você escreveu na primeira parte para que ele implemente somas de prefixos

C/C++

```
/*
 * Compile: mpicc -g -Wall -o mpi_ring_b mpi_ring_b.c
 * Run:      mpiexec -n <p> ./mpi_ring_b
 */
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char* argv[]) {
    int my_rank, comm_sz;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

    srand(my_rank + 10);
    int my_val = rand() % 20;

    int dest    = (my_rank + 1) % comm_sz;
    int source  = (my_rank - 1 + comm_sz) % comm_sz;
    int sendtag = 0, recvtag = 0;

    /* Prefix sum via anel: na iteração i, temp_val contém o valor
     * originado do processo (my_rank - i) mod p.
     * Acumula apenas se esse processo tem rank < my_rank. */
    int temp_val = my_val;
    int prefix   = my_val;

    for (int i = 1; i < comm_sz; i++) {
        MPI_Sendrecv_replace(&temp_val, 1, MPI_INT,
                             dest, sendtag,
                             source, recvtag,
                             MPI_COMM_WORLD, &status);
        int sender_rank = (my_rank - i + comm_sz) % comm_sz;
        if (sender_rank < my_rank)
            prefix += temp_val;
    }

    printf("Proc %d: my_val=%d prefix_sum=%d\n", my_rank, my_val,
    prefix);
    fflush(stdout);

    MPI_Finalize();
}
```

```
    return 0;
}
```

7. As funções `MPI_Scatter` e `MPI_Gather` têm a limitação de que cada processo deve enviar ou receber o mesmo número de itens de dados. Quando este não for o caso, devemos utilizar as funções `MPI_Gatherv` e `MPI_Scatterv`. Consulte a documentação dessas funções e modifique seu programa da questão 4 para que ele possa lidar corretamente com o caso quando `n` não é divisível por `comm_sz`.

C/C++

```
/*
 * Compile: mpicc -g -Wall -o mpi_vec_q7 mpi_vec_q7.c -lm
 * Run:     mpiexec -n <p> ./mpi_vec_q7
 */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <mpi.h>

int main(void) {
    int my_rank, comm_sz, n;
    double scalar;
    double *x = NULL, *y = NULL;
    double *local_x, *local_y, *result_x = NULL;
    double local_norm_sq, global_norm_sq;

    int *sendcounts = NULL, *displs = NULL;
    int local_n;

    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

    if (my_rank == 0) {
        printf("Digite n (qualquer valor): ");
        scanf("%d", &n);
        printf("Digite o escalar: ");
        scanf("%lf", &scalar);

        x = malloc(n * sizeof(double));
        y = malloc(n * sizeof(double));
```

```

    printf("Digite os %d elementos do vetor x:\n", n);
    for (int i = 0; i < n; i++) scanf("%lf", &x[i]);
    printf("Digite os %d elementos do vetor y:\n", n);
    for (int i = 0; i < n; i++) scanf("%lf", &y[i]);
}

MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(&scalar, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);

/* Calcula sendcounts e displs para distribuição não uniforme
*/
sendcounts = malloc(comm_sz * sizeof(int));
displs = malloc(comm_sz * sizeof(int));
int base = n / comm_sz, rem = n % comm_sz;
int offset = 0;
for (int q = 0; q < comm_sz; q++) {
    sendcounts[q] = (q < rem) ? base + 1 : base;
    displs[q] = offset;
    offset += sendcounts[q];
}
local_n = sendcounts[my_rank];

local_x = malloc(local_n * sizeof(double));
local_y = malloc(local_n * sizeof(double));

MPI_Scatterv(x, sendcounts, displs, MPI_DOUBLE,
            local_x, local_n, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Scatterv(y, sendcounts, displs, MPI_DOUBLE,
            local_y, local_n, MPI_DOUBLE, 0, MPI_COMM_WORLD);

for (int i = 0; i < local_n; i++) local_x[i] *= scalar;

local_norm_sq = 0.0;
for (int i = 0; i < local_n; i++) local_norm_sq += local_y[i]
* local_y[i];

if (my_rank == 0) result_x = malloc(n * sizeof(double));
MPI_Gatherv(local_x, local_n, MPI_DOUBLE,
            result_x, sendcounts, displs, MPI_DOUBLE, 0,
MPI_COMM_WORLD);

MPI_Reduce(&local_norm_sq, &global_norm_sq, 1, MPI_DOUBLE,
MPI_SUM, 0, MPI_COMM_WORLD);

if (my_rank == 0) {
    printf("\nResultado de x * %.2f:\n", scalar);
    for (int i = 0; i < n; i++) printf("%.4f ", result_x[i]);
    printf("\nNorma de y: %.6f\n", sqrt(global_norm_sq));
    free(x); free(y); free(result_x);
}

```

```

    }

    free(local_x); free(local_y);
    free(sendcounts); free(displs);
    MPI_Finalize();
    return 0;
}

```

8. Suponha que $\text{comm_sz} = 8$ e o vetor $x = (0, 1, 2, \dots, 15)$ tenha sido distribuído entre os processos usando uma distribuição em bloco. Desenhe um diagrama ilustrando as etapas de uma implementação borboleta da função `allgather` de x .

Processo	Componentes
0	0 e 1
1	2 e 3
2	4 e 5
3	6 e 7
4	8 e 9
5	10 e 11
6	12 e 13
7	14 e 15

O `Allgather` coletivo em borboleta funciona em $\log_2(8) = 3$ fases.
Em cada fase, processos parceiros trocam seus dados acumulados até então.

Proc	Inicial	Fase 1 (stride=1)	Fase 2 (stride=2)	Fase 3 (stride=4)
P0	{0,1}	{0..3}	{0..7}	{0..15}
P1	{2,3}	{0..3}	{0..7}	{0..15}
P2	{4,5}	{4..7}	{0..7}	{0..15}
P3	{6,7}	{4..7}	{0..7}	{0..15}
P4	{8,9}	{8..11}	{8..15}	{0..15}
P5	{10,11}	{8..11}	{8..15}	{0..15}
P6	{12,13}	{12..15}	{8..15}	{0..15}

Proc	Inicial	Fase 1 (stride=1)	Fase 2 (stride=2)	Fase 3 (stride=4)
P7	{14,15}	{12..15}	{8..15}	{0..15}

9. A função `MPI_Type_contiguous` pode ser usada para construir um tipo de dados derivado de uma coleção de elementos contíguos em uma matriz. Sua sintaxe é

```
int MPI_Type_contiguous(
int count /* in */,
MPI_Datatype old_mpi_t /* in */,
MPI_Datatype* new_mpi_t_p /* out */);
```

Modifique as funções `Read_vector` e `Print_vector` (`mpi_vector_add.c`) para que elas usem um tipo de dados MPI criado por uma chamada para `MPI_Type_contiguous` e um argumento de contagem de 1 nas chamadas para `MPI_Scatter` e `MPI_Gather`.

C/C++

`Read_vector:`

```
void Read_vector(double local_a[], int local_n, int n, char
vec_name[],
                int my_rank, MPI_Comm comm) {
    double* a = NULL;
    int local_ok = 1;

    MPI_Datatype contig_type;
    MPI_Type_contiguous(local_n, MPI_DOUBLE, &contig_type);
    MPI_Type_commit(&contig_type);

    if (my_rank == 0) {
        a = malloc(n * sizeof(double));
        if (a == NULL) local_ok = 0;
        Check_for_error(local_ok, "Read_vector", "malloc falhou",
comm);
        printf("Entre com o vetor %s\n", vec_name);
        for (int i = 0; i < n; i++) scanf("%lf", &a[i]);
        MPI_Scatter(a, 1, contig_type, local_a, 1, contig_type, 0,
comm);
        free(a);
    } else {
        Check_for_error(local_ok, "Read_vector", "malloc falhou",
comm);
        MPI_Scatter(a, 1, contig_type, local_a, 1, contig_type, 0,
comm);
    }

    MPI_Type_free(&contig_type);
```



```
}
```

Print_vector:

```
void Print_vector(double local_b[], int local_n, int n, char
title[],
                int my_rank, MPI_Comm comm) {
    double* b = NULL;
    int local_ok = 1;

    MPI_Datatype contig_type;
    MPI_Type_contiguous(local_n, MPI_DOUBLE, &contig_type);
    MPI_Type_commit(&contig_type);

    if (my_rank == 0) {
        b = malloc(n * sizeof(double));
        if (b == NULL) local_ok = 0;
        Check_for_error(local_ok, "Print_vector", "malloc falhou",
comm);
        MPI_Gather(local_b, 1, contig_type, b, 1, contig_type, 0,
comm);
        printf("%s\n", title);
        for (int i = 0; i < n; i++) printf("%f ", b[i]);
        printf("\n");
        free(b);
    } else {
        Check_for_error(local_ok, "Print_vector", "malloc falhou",
comm);
        MPI_Gather(local_b, 1, contig_type, b, 1, contig_type, 0,
comm);
    }

    MPI_Type_free(&contig_type);
}
```

10. A função `MPI_Type_indexed` pode ser usada para construir um tipo de dados derivado de elementos arbitrários de um vetor. Sua sintaxe é

```
int MPI_Type_indexed(
int count /* in */,
int array_of_blocklengths[] /* in */,
int array_of_displacements[] /* in */,
MPI_Datatype old_mpi_t /* in */,
MPI_Datatype* new_mpi_t_p /* out */);
```

Ao contrário da função `MPI_Type_create_struct`, os deslocamentos são medidos em unidades de `old_mpi_t` - não em bytes. Use a função `MPI_Type_indexed` para criar um tipo de dados derivado que corresponda à parte triangular superior de uma matriz quadrada.

Por exemplo, na matriz 4 x 4

```
0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15
```

a parte triangular superior são os elementos 0, 1, 2, 3, 5, 6, 7, 10, 11, 15. O processo 0 deve ler uma matriz $n \times n$ como um vetor unidimensional, criar o tipo de dados derivado e enviar a parte triangular superior com uma única chamada de MPI_Send. O processo 1 deve receber a parte triangular superior com uma única chamada ao MPI_Recv e depois imprimir os dados recebidos.

C/C++

```
/*
 * Compile: mpicc -g -Wall -o mpi_upper_q10 mpi_upper_q10.c
 * Run:      mpiexec -n 2 ./mpi_upper_q10
 */
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(void) {
    int my_rank, comm_sz;
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

    int n;
    int *matrix = NULL;

    if (my_rank == 0) {
        printf("Digite n (ordem da matriz): ");
        scanf("%d", &n);
    }
    MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);

    int upper_count = n * (n + 1) / 2;

    int *blocklengths = malloc(n * sizeof(int));
    int *displacements = malloc(n * sizeof(int));

    for (int i = 0; i < n; i++) {
        blocklengths[i] = n - i;
        displacements[i] = i * n + i;
    }

    MPI_Datatype upper_tri_type;
    MPI_Type_indexed(n, blocklengths, displacements, MPI_INT,
        &upper_tri_type);
```

```

MPI_Type_commit(&upper_tri_type);

if (my_rank == 0) {
    matrix = malloc(n * n * sizeof(int));
    printf("Digite os %d elementos (row-major):\n", n * n);
    for (int i = 0; i < n * n; i++) scanf("%d", &matrix[i]);
    MPI_Send(matrix, 1, upper_tri_type, 1, 0, MPI_COMM_WORLD);
    printf("Processo 0: triangular superior enviada.\n");
    free(matrix);
} else if (my_rank == 1) {
    int *recv_buf = malloc(upper_count * sizeof(int));
    MPI_Recv(recv_buf, upper_count, MPI_INT, 0, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Processo 1 recebeu %d elementos:\n", upper_count);
    for (int i = 0; i < upper_count; i++) printf("%d ",
recv_buf[i]);
    printf("\n");
    free(recv_buf);
}

MPI_Type_free(&upper_tri_type);
free(blocklengths);
free(displacements);
MPI_Finalize();
return 0;
}

```

11. As funções `MPI_Pack` e `MPI_Unpack` fornecem uma alternativa aos tipos de dados derivados para agrupar dados. O `MPI_Pack` copia os dados a serem enviados, um bloco por vez, em um buffer fornecido pelo usuário. O buffer pode então ser enviado e recebido. Após o recebimento dos dados, `MPI_Unpack` pode ser usado para descompactá-los do buffer de recebimento. A sintaxe do `MPI_Pack` é

```

int MPI_Pack(
void* in_buf /* in */,
int in_buf_count /* in */,
MPI_Datatype datatype /* in */,
void* pack_buf /* out */,
int pack_buf_sz /* in */,
int* position_p /* in/out */,
MPI_Comm comm /* in */);

```

Poderíamos, portanto, empacotar os dados de entrada para o programa da regra dos trapézios com o seguinte código:

```

char pack_buf[100];
int position = 0;
MPI_Pack(&a, 1, MPI_DOUBLE, pack_buf, 100, &position, comm);
MPI_Pack(&b, 1, MPI_DOUBLE, pack_buf, 100, &position, comm);
MPI_Pack(&n, 1, MPI_INT, pack_buf, 100, &position, comm);

```

A chave é o argumento da position. Quando MPI_Pack é chamado, a posição deve referir-se ao primeiro slot disponível no pack_buf. Quando MPI_Pack retorna, ele se refere ao primeiro slot disponível após os dados que acabaram de ser compactados, portanto, após o processo 0 executar este código, todos os processos podem chamar MPI_Bcast:

```
MPI_Bcast(pack_buf, 100, MPI_PACKED, 0, comm);
```

Observe que o tipo de dados MPI para um buffer compactado é MPI_PACKED. Agora os outros processos podem descompactar os dados usando: MPI_Unpack:

```
int MPI_Unpack(
void* pack_buf /* in */,
int pack_buf_sz /* in */,
int* position_p /* in/out */,
void* out_buf /* out */,
int out_buf_count /* in */,
MPI_Datatype datatype /* in */,
MPI_Comm comm /* in */);
```

MPI_Unpack pode ser usado "invertendo" as etapas do MPI_Pack, ou seja, os dados são descompactados um bloco por vez, começando em position = 0.

Escreva outra função Get_input para o programa da regra dos trapézios. Este deve usar MPI_Pack no processo 0 e MPI_Unpack nos demais processos.

C/C++

```
void Get_input(int my_rank, int comm_sz, double* a_p, double* b_p,
int* n_p) {
    char pack_buf[100];
    int position = 0;

    if (my_rank == 0) {
        printf("Entre com a, b e n:\n");
        scanf("%lf %lf %d", a_p, b_p, n_p);

        MPI_Pack(a_p, 1, MPI_DOUBLE, pack_buf, 100, &position,
MPI_COMM_WORLD);
        MPI_Pack(b_p, 1, MPI_DOUBLE, pack_buf, 100, &position,
MPI_COMM_WORLD);
        MPI_Pack(n_p, 1, MPI_INT, pack_buf, 100, &position,
MPI_COMM_WORLD);
    }

    MPI_Bcast(pack_buf, 100, MPI_PACKED, 0, MPI_COMM_WORLD);

    if (my_rank != 0) {
        position = 0;
        MPI_Unpack(pack_buf, 100, &position, a_p, 1, MPI_DOUBLE,
MPI_COMM_WORLD);
        MPI_Unpack(pack_buf, 100, &position, b_p, 1, MPI_DOUBLE,
MPI_COMM_WORLD);
        MPI_Unpack(pack_buf, 100, &position, n_p, 1, MPI_INT,
MPI_COMM_WORLD);
    }
}
```

```
}  
}
```

14. Modifique a ordenação ímpar-par paralela (mpi_odd_even.c) para que as funções Merge simplesmente troquem os ponteiros do vetor após encontrar os elementos menores ou maiores. Que efeito essa mudança tem no tempo de execução geral? Fixe uma quantidade de processos (ex.: 8 processos) e analise a influência do tamanho dos vetores no tempo de execução.

```
C/C++  
void Merge_low(int** local_A_pp, int temp_B[], int temp_C[], int  
local_n) {  
    /* Merge normal: os local_n menores vão para temp_C */  
    int ai=0, bi=0, ci=0;  
    while (ci < local_n) {  
        if (ai < local_n && (bi >= local_n || (*local_A_pp)[ai] <=  
temp_B[bi]))  
            temp_C[ci++] = (*local_A_pp)[ai++];  
        else  
            temp_C[ci++] = temp_B[bi++];  
    }  
    /* TROCA DE PONTEIRO: sem memcpy! */  
    int* tmp = *local_A_pp;  
    *local_A_pp = temp_C;  
    /* (tmp agora é o buffer antigo, disponível para reutilização)  
*/  
}
```